Department of Electrical and Computer Engineering

Course: Interfacing Techniques (ENCS4380)

Homework: Advanced Non-Blocking Multitasking & Microcontroller Interfacing

Instructor: Dr. Wasel Ghanem

August 21, 2026

Homework Overview

Course:	Microcontroller Systems / Interfacing Techniques
Platform:	ATmega328P (Arduino Uno)
Simulation:	Wokwi / TinkerCad / Proteus / Hardware
Total Marks:	100 pts (5×20 pts)

Core Focus: Advanced multi-tasking timing scenarios centered around non-blocking software architectures using `millis()`, direct AVR bit-masking register operations, register-level hardware interrupts, and real-time state machine implementations.

—

General Instructions

- You may simulate in Wokwi, TinkerCad, or Proteus, then run on physical AVR microcontrollers.
- **Banned Functions:** Unless explicitly allowed, standard high-level functions like `delay()`, `delayMicroseconds()`, `digitalWrite()`, `digitalRead()`, and `pinMode()` are strictly prohibited across all problems. Using them voids marks for that specific section.
- **Direct AVR Register Access:** Configure pin directions with `DDRx`, output latch state with `PORTx`, and input buffer state with `PINx`.
- Always use Read-Modify-Write bitwise masking (`|=`, `&=` , `^=`) to avoid corrupting unrelated pin state registers.
- Submit one compiled `.ino` code file per question, circuit schematic/wiring screenshots, and a half-page engineering report detailing timing execution and register calculations.
- Late policy: 10% penalty per day late (maximum 3 days allowed).

—

1 Question 1 — Bare-Metal HD44780 LCD Driver (No Library, Port Masking)

In this task, you will build a bare-metal display driver for an HD44780 LCD controller operating in 4-bit data bus mode using direct AVR bit manipulation.

Hardware Pin Mapping (Mandatory)

LCD Pin	ATmega328P Connection	Register Bit Assignment
RS	PB0 (Arduino D8)	Bit 0 of `PORTB`
EN	PB1 (Arduino D9)	Bit 1 of `PORTB`
RW	GND	Write-Only Mode
D4–D7	PD4–PD7 (Arduino D4–D7)	Bits 4–7 of `PORTD`
VSS/VDD/VO	GND / +5V / 10 kΩ Potentiometer	Power & Contrast Control

Functions to Implement

- `void lcd_init(void)`: Execute the standard 4-bit cold-boot initialization sequence specified in the HD44780 datasheet.
- `void lcd_command(uint8_t cmd)`: Transmit a command byte as two distinct 4-bit nibbles with RS = 0.
- `void lcd_data(uint8_t b)`: Transmit a data byte as two distinct 4-bit nibbles with RS = 1.
- `void lcd_setCursor(uint8_t col, uint8_t row)`: Set DDRAM address (Row 0 → 0x00, Row 1 → 0x40).
- `void lcd_print(const char *s)`: Write a null-terminated string to the display.

Demo Program & Requirements

- **Row 0:** Display static string: `"[Your Full Name] - ID"`.
- **Row 1:** Display a multi-tasking millisecond runtime counter updated continuously via non-blocking `millis()` logic without freezing execution.
- Use `micros()` only for mandatory HD44780 timing holds ($\geq 40\,\mu$s execution delays, $\geq$ 1.64 ms clear command delay). Standard `delay()` calls are forbidden.

—

2 Question 2 — Non-Blocking Multi-Pattern Timing & Debouncing via `millis()`

Construct an asymmetric, multi-channel pattern generator using non-blocking `millis()` scheduling loops running simultaneously alongside a debounced button interface.

Task Specification

- **LED A (PB0):** Blinks continuously with an asymmetric profile: **200 ms ON, 800 ms OFF**.
- **LED B (PB1):** Blinks symmetrically every **350 ms**.
- **LED C (PB2):** Phase-shifted pattern: Starts in an **ON** state at boot, remains ON for **150 ms**, then turns OFF for **650 ms**.

- **Button 1 (PD2):** Configure with internal pull-up resister (`PORTD |= (1 << PD2)`). Read pin state directly from `PIND`. Pressing the button toggles **LED D (PB3)**.
- **Challenge - Non-Blocking Debounce & Press Duration:** Implement a strict non-blocking state machine for Button 1:
  - Ignore mechanical contact bounces shorter than **40 ms**.
  - Measure the active hold duration. If the button is held continuously for **¿ 2000 ms**, trigger a fast safety flash pattern on **LED D** (50 ms toggle rate) until released.
- **Serial Monitor:** Print system status over UART every 2.5 seconds using non-blocking timing: `"A:<state> B:<state> C:<state> D:<state> Hold:<duration>ms"`.

Bonus Challenge (+3 pts)

Add a fifth LED on **PB4**. Turn this LED ON **only** if LEDs A, B, and C are simultaneously in the HIGH state. Determine this state within a single bitwise masking operation evaluated against `PORTB`.

—

3 Question 3 — Dynamic Task Scheduling & Timer Library Coordination

Utilize the `Timer` library (`Timer.h`) to coordinate hardware events while driving performance display output through your bare-metal LCD driver from Question 1.

Task Specification

- **LED 1 (PB0):** Configure `t1.oscillate(LED1_PIN, 250, HIGH)` to toggle state every 250 ms.
- **LED 2 (PB1):** Fire a dynamic pulse of 150 ms every 3 seconds triggered via `t2.every(3000, triggerPulse)`.
- **System Heartbeat:** Increment an execution tick counter every 1000 ms using a background timer callback.
- **LCD Refresh:** Update LCD Row 0 every 250 ms displaying `"Ticks: <cnt>"`.
- **Challenging Timing Scenario:** Monitor LED 2's activity. When a 150 ms pulse fires, display `"PULSE: ACTIVE"` on LCD Row 1. Once the pulse expires, keep the LCD Row 1 message reading `"PULSE: EXPIRED"` for an additional **450 ms window** using pure `millis()` timestamp tracking without allocating additional software timers or calling delays.

Bonus Challenge (+2 pts)

Wire a mode toggle button to `PD3`. When pressed, temporarily double the frequency of all system timers (halve period durations) for a **3-second window**, returning back to baseline parameters automatically without destroying active system state machine references.

—

4 Question 4 — Direct Register External Interrupts vs. `attachInterrupt()`

Explore hardware interrupts by designing a dual-channel high-speed pulse counter comparing Arduino abstraction layers against raw AVR register configurations.

Part A — Standard API Implementation (10 pts)

- Attach Button 1 to `INT0` (`D2`) and Button 2 to `INT1` (`D3`).
- Trigger interrupts on the **FALLING** signal edge.
- **INT0 ISR:** Increment an integer counter `pulse_count`.
- **INT1 ISR:** Reset `pulse_count` to zero.
- Implement non-blocking ISR software debouncing using a `millis()` guard threshold ($\geq$ 50 ms between accepted hardware edge transitions).
- Display updated counts on the bare-metal LCD and send an asynchronous event notification through Serial.

Part B — Pure Register-Level Hardware Configuration (10 pts)

Remove all `attachInterrupt()` calls and replace them with raw register configuration manipulation:

1. Configure external interrupt control registers `EICRA` and `EIMSK` directly to capture falling edge signals on `INT0` and `INT1`.
2. Implement interrupt service routines using raw vector definitions: `ISR(INT0_vect)` and `ISR(INT1_vect)`.
3. Maintain complete non-blocking constraints: **No `Serial.print()` calls inside ISR routines**. Set volatile flag variables within the ISR and process log outputs within the main execution loop.
4. Document the precise bit manipulation masks used for `EICRA` and `EIMSK` in your engineering report.

—

5 Question 5 — Integrated Challenge: Advanced Multi-State Traffic Controller

Integrate bare-metal bit-masking, complex non-blocking `millis()` timing checks, register-level interrupts, and custom display driving into an automated industrial traffic and pedestrian crossing controller.

System Architecture & State Machine Specification

- **Traffic Signal LEDs (PORTB):** Red (`PB0`), Yellow (`PB1`), Green (`PB2`), Pedestrian Walk (`PB3`). Controlled strictly via bit manipulation.
- **Pedestrian Request Button (`INT0` / `D2`):** Configured at register level (`EICRA`/`EIMSK`). Pressing the button sets a `volatile bool pedRequest` flag.

State	Nominal Time	LED Status	LCD Output (Row 0)
GREEN	6000 ms	Green ON	"TRAFFIC: GO"
YELLOW	2500 ms	Yellow ON	"TRAFFIC: SLOW"
RED	5000 ms	Red ON	"TRAFFIC: STOP"
PED_CROSS	4000 ms	Red ON + Pedestrian LED Flashes	"PED: CROSSING"

State Transition Challenging Timing Mechanics

Complex Non-Blocking Rules

- **Dynamic Pedestrian Interruption:**
  - If pedRequest occurs during GREEN phase and elapsed green time is < 3000 ms, hold GREEN until $t = 3000$ ms, then force an immediate transition to YELLOW.
  - If pedRequest occurs after 3000 ms of GREEN state execution, transition immediately to YELLOW.
  - If pedRequest occurs during RED state, transition directly to PED_CROSS immediately upon conclusion of the current RED cycle.
  - Ignore pedestrian interrupts recorded during active PED_CROSS cycles.
- **Pedestrian Signal Blinking:** During PED_CROSS, flash the Pedestrian LED on PB3 at 2.5 Hz using a non-blocking millis() toggle loop.
- **Real-Time LCD Display:** LCD Row 1 must dynamically update every 200 ms displaying remaining state duration formatted as: "Rem Time: X.X s".

Bonus Challenge (+4 pts)

Multiplex a single 7-segment display on PORTD showing a live numeric countdown ($9 \rightarrow 0$) corresponding to phase completion. Drive the 7-segment display directly using bitwise operations with numeric digit lookup tables stored in PROGMEM.

—

6 Submission Requirements

- **ZIP Archive Naming Convention:** LastName_FirstName_HW1.zip
- **Folder Structure:** Provide five independent sub-directories (Q1 through Q5) containing:
  1. Source code file (.ino).
  2. Wiring schematic or Wokwi / TinkerCad simulation screenshot.
  3. Half-page technical PDF report detailing register bitwise configurations, timing diagram calculations, and system behavior.
- **Demonstration Video:** A single video submission ($\leq$ 5 minutes) demonstrating functional operation across all 5 problems.

—

7 Starter Code Snippet: Direct Register & Masking Definitions

To assist in bare-metal setup, use the following code skeleton for bit operations:

```c
#include <avr/io.h>
#include <avr/interrupt.h>

#define LCD_DATA_MASK 0xF0  // PD4..PD7
#define RS_BIT        (1 << PB0)
#define EN_BIT        (1 << PB1)

void lcd_pulse_en(void) {
    PORTB |= EN_BIT;
    asm volatile("nop\n nop\n nop\n nop\n"); // >= 450 ns pulse delay
    PORTB &= ~EN_BIT;
}

void lcd_send_nibble(uint8_t nibble) {
    // Preserve lower bits on PORTD while updating upper data pins
    PORTD = (PORTD & ~LCD_DATA_MASK) | (nibble & LCD_DATA_MASK);
    lcd_pulse_en();
}

void setup_registers(void) {
    // Configure PB0, PB1 as Outputs without affecting other pins
    DDRB |= RS_BIT | EN_BIT;

    // Configure PD4..PD7 as Outputs
    DDRD |= LCD_DATA_MASK;
}
```